

Lab 6

Dual BCD Up/Down Counters, ALU, and Control Display on 7-Segment

By: Ben Robles & Robert Stevenson
Group U

ECE-3300L
Summer 2026

Date: 07/23/2026

Design:

clock_divider.v:

```
module clock_divider (
    input wire clk,
    output reg [31:0] clk_div
);

    initial clk_div = 0;

    always@(posedge clk)
    begin
        clk_div <= clk_div + 1;
    end

endmodule
```

Description:

This module creates a 32-bit counter that increments on posedge clk. This functions as a clock divider because each bit toggles at different frequencies in a way that higher bits toggle slower.

mux2x1.v:

```
module mux2x1(
    input wire sel,
    input wire a,
    input wire b,
    output wire y
);

    wire selnot;
    assign selnot = ~sel;
    assign y = (a & selnot) | (b & sel);

endmodule
```

mux32x1.v:

```
module mux32x1(
    input wire [31:0] in,
    input wire [4:0] sel,
    output wire out
);

    wire [15:0] level1;
    wire [7:0] level2;
    wire [3:0] level3;
    wire [1:0] level4;
```

```

genvar i;
generate
  for(i=0; i<16; i=i+1)begin
    mux2x1 M1(.a(in[i*2]), .b(in[i*2+1]), .sel(sel[0]), .y(level1[i]));
  end
  for(i=0; i<8; i=i+1)begin
    mux2x1 M2(.a(level1[2*i]), .b(level1[2*i+1]), .sel(sel[1]), .y(level2[i]));
  end
  for(i=0; i<4; i=i+1)begin
    mux2x1 M3(.a(level2[2*i]), .b(level2[2*i+1]), .sel(sel[2]), .y(level3[i]));
  end
  for(i=0; i<2; i=i+1)begin
    mux2x1 M4(.a(level3[2*i]), .b(level3[2*i+1]), .sel(sel[3]), .y(level4[i]));
  end
  mux2x1 M5(.a(level4[0]), .b(level4[1]), .sel(sel[4]), .y(out));
endgenerate
endmodule

```

Description:

This 32x1 mux is used as a selector for the different frequencies output by the clock divider

debounce.v:

```

module debounce(
    input wire clk,
    input wire btn_raw,
    output reg btn_clean
);

reg[2:0] state;
initial state = 3'b000;

always @(posedge clk)begin
    state = {state[1:0], btn_raw};
    if(state == 3'b000) btn_clean = 0;
    else if (state == 3'b111) btn_clean = 1;
end
endmodule

```

Description:

This module debounces the reset button by verifying input stability for 3 cycles

bcd_counter.v:

```

module bcd_counter(
    input clk,
    input rst_n,
    input dir,
    output reg [3:0] digit
);

always@(posedge clk or negedge rst_n) begin

```

```

if (lrst_n) begin
    digit <= 4'd0;
end
else begin
    if (dir) begin
        if (digit == 4'd9)
            digit <= 4'd0;
        else
            digit <= digit + 4'd1;
        end
    else begin
        if (digit == 4'd0)
            digit <= 4'd9;
        else
            digit <= digit - 4'd1;
        end
    end
end
end
endmodule

```

Description:

This single digit BCD up/down counter activates every clock cycle and rollover 0 to 9

alu.v:

```

module alu(
    input [3:0] a,
    input [3:0] b,
    input [1:0] ctrl,
    output reg [7:0] result
);

always@(*) begin
    case(ctrl)
        2'b00: result = {4'b0000, a} + {4'b0000, b};
        2'b01: result = {4'b0000, a} - {4'b0000, b};
        default: result = 8'b0;
    endcase
end

endmodule

```

Description:

This ALU design operates on two unsigned 4-bit integers. The output is 8-bits with no carry and wraps on underflow. Operation is controlled by 2 bits (00=add, 01=sub)

control_decoder.v:

```

module control_decoder(
    input [3:0] inp,
    output wire [3:0] ctrl

```

```

    );
    assign ctrl = inp;

endmodule

```

Description:

This module bundles four control bits into an output nibble

seg7_scan.v:

```

module seg7_scan(
    input clk,
    input rst_n,
    input [7:0] result,
    input [3:0] ctrl_nibble,
    output reg [7:0] AN,
    output reg [6:0] SEG
);

    reg [19:0] temp;
    reg [3:0] digit;

    wire [1:0] s;
    assign s = temp[18:17];

    always@(posedge clk)
    begin
        if (!rst_n)
            temp <= 0;
        else
            temp <= temp + 1;
    end

    always@(*)
    begin
        case(s)
            2'b00: begin
                digit = result[3:0];
                AN = 8'b11111110;
            end

            2'b01: begin
                digit = result[7:4];
                AN = 8'b11111101;
            end

            2'b10: begin
                digit = ctrl_nibble;
                AN = 8'b11111011;
            end

            default: begin
                digit = 0;
                AN = 8'b11111111;
            end
        endcase
    end

```

```

end

always @(*)
begin
    case(digit)
        4'd0: SEG=7'b0000001;
        4'd1: SEG=7'b1001111;
        4'd2: SEG=7'b0010010;
        4'd3: SEG=7'b0000110;
        4'd4: SEG=7'b1001100;
        4'd5: SEG=7'b0100100;
        4'd6: SEG=7'b0100000;
        4'd7: SEG=7'b0001111;
        4'd8: SEG=7'b0000000;
        4'd9: SEG=7'b0001100;
        4'd10: SEG=7'b0001000;
        4'd11: SEG=7'b1100000;
        4'd12: SEG=7'b0110001;
        4'd13: SEG=7'b1000010;
        4'd14: SEG=7'b0110000;
        4'd15: SEG=7'b0111000;
        default: SEG=7'b1111111;
    endcase
end
endmodule

```

Description:

This seven segment controller time multiplexes bit 17 and 18 of a 20 bit synchronous delay register to drive three 7 segment displays with hexadecimal digits

top_lab6.v:

```

module top_lab6(
    input CLK,
    input [8:0] SW,
    input BTNC,
    output wire [7:0] AN,
    output wire [6:0] SEG,
    output wire [7:0] LED
);

    wire rst, rst_n;
    assign rst_n = ~rst;

    wire[31:0] counter;
    wire slow_clk;

    wire [3:0] ctrl_nibble;
    wire [3:0] count_ones;
    wire [3:0] count_tens;
    wire [7:0] result;

    clock_divider clock_division(
        .clk(CLK),
        .clk_div(counter)
    );

```

```

mux32x1 counter_muxplexer(
    .in(counter),
    .sel(~SW[4:0]),
    .out(slow_clk)
);

debounce debounce_rst(
    .btn_raw(BTNC),
    .btn_clean(rst),
    .clk(CLK)
);

control_decoder ctrl_out (
    .inp(SW[8:5]),
    .ctrl(ctrl_nibble)
);

bcd_counter counter_ones (
    .clk(slow_clk),
    .rst_n(rst_n),
    .dir(SW[7]),
    .digit(count_ones)
);

bcd_counter counter_tens (
    .clk(slow_clk),
    .rst_n(rst_n),
    .dir(SW[8]),
    .digit(count_tens)
);

assign LED = {count_tens, count_ones};

alu bcd_alu (
    .ctrl(SW[6:5]),
    .a(count_ones),
    .b(count_tens),
    .result(result)
);

seg7_scan seg7_out(
    .clk(CLK),
    .rst_n(rst_n),
    .result(result),
    .ctrl_nibble(ctrl_nibble),
    .AN(AN),
    .SEG(SEG)
);

endmodule

```

Description:

The top module is very simple, it instantiates each module and wires them up. Besides instantiation, the LEDs are assigned to display each value of the counter. The only other thing of note is the reversal of the sw input into the clock divider mux, this was done to match the given

hardware interface where the speed select index is 0=slow, 31=fast (because the clock divider we generated has higher bits toggle at lower frequencies)

XDC Snippet:

```
## Clock signal
set_property -dict { PACKAGE_PIN E3   IOSTANDARD LVCMOS33 } [get_ports { CLK}];
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { CLK}];

##Switches
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { SW[0] }];
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { SW[1] }];
...
set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { SW[7] }];
set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { SW[8] }];

## LEDs
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { LED[0] }];
set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { LED[1] }];
...
set_property -dict { PACKAGE_PIN U17   IOSTANDARD LVCMOS33 } [get_ports { LED[6] }];
set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports { LED[7] }];

#7 segment display

set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { SEG[6] }];
set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports { SEG[5] }];
...
set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports { SEG[1] }];
set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports { SEG[0] }];

set_property -dict { PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { AN[0] }];
set_property -dict { PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { AN[1] }];
...
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { AN[6] }];
set_property -dict { PACKAGE_PIN U13   IOSTANDARD LVCMOS33 } [get_ports { AN[7] }];

##Buttons

set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { BTNC }]; #IO_L9P_T1_DQS_14 Sch=btnc
```

Simulation:

clock_divider_tb.v:

```
module clock_divider_tb (
    );
```



```

reg clk_tb;
wire [31:0] clk_div_tb;

integer i;
integer count; // Expected clk_div_tb count

integer dn1, dn2; // delta nibble clk_div_tb[3:0], delta nibble clk_div_tb[7:4]
integer clk_prev; // clk_div_tb prev

clock_divider dut (
    .clk(clk_tb),
    .clk_div(clk_div_tb)
);

initial begin: Initialize
    clk_tb = 0;
    count = 0;
    dn1 = 0;
    dn2 = 0;
    clk_prev = 0;
end

always begin: Clock
    #5 clk_tb = ~clk_tb;
end

initial begin: Testbench
    #1
    for(i = 0; i < 128; i = i + 1) begin
        clk_prev = clk_div_tb[7:0]; // Set prev

        @(posedge clk_tb)
        count = count + 1; // Update count after clk edge

        #1
        if(count != clk_div_tb) begin // Fail if count != clk_div_tb
            $display("FAIL CLK = %b, CLK_DIV = %b", count, clk_div_tb);
            $fatal;
        end

        if(clk_div_tb[3:0] != clk_prev[3:0]) // Track delta nibble 1
            dn1 = dn1 + 1;

        if(clk_div_tb[7:4] != clk_prev[7:4]) // Track delta nibble 2
            dn2 = dn2 + 1;
        end
    end
    #5

    $display("PASS delta n1 = %d, delta n2 = %d", dn1, dn2); // Expect dn1 to change 16* faster than dn2
    $finish;
end

endmodule

```

Description:

To test the clock divider, I used a for loop to automatically verify it increments properly with each clock edge. I also kept track of how many times the first and second nibble of the clock divider

changed to verify they changed at different frequencies. I found that the first nibble toggles 16 times faster than the second, confirming the implementation that higher bits toggle at lower frequencies.

Output:

```
run 12 ms
PASS delta n1 =          128, delta n2 =          8
$finish called at time : 1281 ns : File "D:/School/ECE3300/ECE3300_Lab5_GroupU
```

mux2x1_tb.v

```
module mux2x1_tb(

);

reg a, b, sel;
wire y;
integer tasksfailed, tempfailed;
reg check;
reg [1:0] i, j, k, in;

mux2x1 mux_tb(.a(a), .b(b), .sel(sel), .y(y));

initial begin
    a = 0;
    b = 0;
    sel = 0;
    tasksfailed = 0;
    tempfailed = 0;

    for(i=0; i<2; i=i+1)begin
        sel = i;
        for(j=0; j<2; j=j+1)begin
            a = j;
            for(k=0; k<2; k=k+1)begin
                b = k;
                in = {b, a};
                #10;
                check = y;
                if(check != in[sel]) tasksfailed = tasksfailed + 1;
                if(tasksfailed > tempfailed) $display("Test failed at %d, %d, %d", i, j, k);
                tempfailed = tasksfailed;
            end
        end
    end
    end
    $finish;
end
endmodule
```

Description:

This testbench uses nested for loops to check all four possible inputs for both possible selection values and checks the output.

mux32x1_tb.v:

```
module mux32x1_tb(

);

reg[31:0] in;
reg[4:0] sel;
wire out;

mux32x1 mux_tb(.in(in), .sel(sel), .out(out));

integer i, tasksfailed;

initial begin
    in = 32'b0;
    sel = 5'b0;
    tasksfailed = 0;
    for(i=0; i<32; i=i+1)begin
        in[i] = 1;
        #5;
        if(out != in[i]) begin
            tasksfailed = tasksfailed + 1;
            $display("test failed");
        end
        in[i] = 0;
        #5;
        if(out != in[i]) begin
            tasksfailed = tasksfailed + 1;
            $display("test failed");
        end
        sel = sel+1;
    end
    if(tasksfailed != 0) $display("%d tasks failed", tasksfailed); else $display("all tasks succeeded");
    $finish;
end

endmodule
```

Description:

This testbench runs a nested four loop and checks that when the selected input is toggled that the output of the multiplexer does the same.

debounce_tb.v:

```
module debounce_tb(
```

```

    );

reg clk_tb;
reg btn_raw_tb;
wire btn_clean_tb;

debounce db_tb(
    .clk(clk_tb),
    .btn_raw(btn_raw_tb),
    .btn_clean(btn_clean_tb)
);

initial
begin
    clk_tb = 1'b0;
end

always
begin
    #5 clk_tb = ~clk_tb;
end

initial
begin
    btn_raw_tb = 1'b0;          // Test 1: Stable low
    repeat(5) @(posedge clk_tb);
    #1;

    if(btn_clean_tb !== 1'b0)
    begin
        $display("FAIL: btn_clean should be 0 after stable low");
        $fatal;
    end

    btn_raw_tb = 1'b1;          // Test 2: Bounce
    @(posedge clk_tb);
    #1;

    btn_raw_tb = 1'b0;
    @(posedge clk_tb);
    #1;

    if(btn_clean_tb !== 1'b0)
    begin
        $display("FAIL: btn_clean changed after bounce");
        $fatal;
    end

    btn_raw_tb = 1'b1;          // Test 3: Stable high
    repeat(5) @(posedge clk_tb);
    #1;

    if(btn_clean_tb !== 1'b1)
    begin
        $display("FAIL: btn_clean should be 1 after stable high");
        $fatal;
    end

    btn_raw_tb = 1'b0;          // Test 4: Bounce
    @(posedge clk_tb);
    #1;

```

```

    btn_raw_tb = 1'b1;
    @(posedge clk_tb);
    #1;

    if(btn_clean_tb !== 1'b1)
    begin
        $display("FAIL: btn_clean changed after bounce");
        $fatal;
    end

    btn_raw_tb = 1'b0;           // Test 5: Final stable low
    repeat(5) @(posedge clk_tb);
    #1;

    if(btn_clean_tb !== 1'b0)
    begin
        $display("FAIL: btn_clean should be 0 after stable low");
        $fatal;
    end

    $display("Passed all tests");
    $finish;
end
endmodule

```

Description:

This debounce testbench automatically verifies the output of stable low -> bouncing -> stable high -> bouncing -> stable low

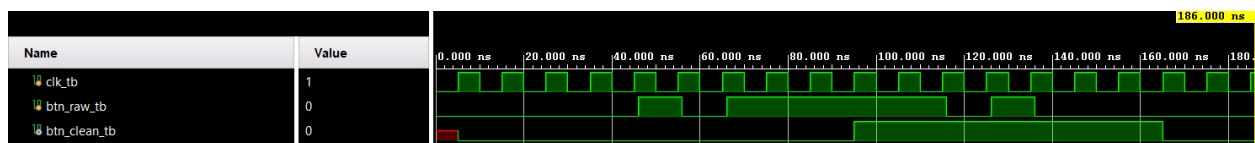
Output:

```

Passed all tests
$finish called at time : 186 ns : File "D:/School/ECE3300/ECE3300 Lab5 GroupU

```

Waveform:



bcd_counter_tb.v:

```

module bcd_counter_tb(

);

reg clk_tb;
reg rst_n_tb;
reg dir_tb;

```

```

wire [3:0] digit_tb;

integer i;
reg [3:0] expected; // Expected digit value

bcd_counter digit_test(
    .clk(clk_tb),
    .rst_n(rst_n_tb),
    .dir(dir_tb),
    .digit(digit_tb)
);

initial begin
    clk_tb = 0;
    rst_n_tb = 0;
    dir_tb = 1;
    expected = 0;
end

always begin
    #5 clk_tb = ~clk_tb;
end

initial begin
    @(posedge clk_tb); // Let digit settle after reset
    #1;
    rst_n_tb = 1;
    #1;
    if(digit_tb !== 4'd0) begin // Test 1: Reset test
        $display("Fail: digit failed to settle on reset    rst_n = %b, digit = %d", rst_n_tb, digit_tb);
        $fatal;
    end
    for(i = 0; i < 10; i = i + 1) begin // Test 2: Incrementing
        @(posedge clk_tb);
        if(expected == 4'd9) begin // Rollover control for expected value
            expected = 4'd0;
        end
        else begin
            expected = expected + 4'd1;
        end
        #1;
        if(digit_tb !== expected) begin // Digit test
            $display("Fail: Digit did not increment properly    digit = %d, expected = %d", digit_tb, expected);
            $fatal;
        end
    end

    expected = 4'd1; // Account for extra up count before switching to down

    @(posedge clk_tb);
    #1;
    dir_tb = 0;
    for(i = 0; i < 12; i = i + 1) begin // Test 3: Decrementing
        @(posedge clk_tb);
        if(expected == 4'd0) begin // Rollover control for expected value
            expected = 4'd9;
        end
        else begin
            expected = expected - 4'd1;
        end
        #1;
    end
end

```

```

        if(digit_tb !== expected) begin // Digit test
            $display("Fail: Digit did not decrement properly    digit = %d, expected = %d", digit_tb, expected);
            $fatal;
        end
    end
end

@(posedge clk_tb);
#1;
rst_n_tb = 0;          // Test 4: Final reset test
@(posedge clk_tb);
#3;
if(digit_tb !== 4'd0) begin
    $display("Fail: Reset did not reset to zero    digit = %d, rst_n = %b", digit_tb, rst_n_tb);
    $fatal;
end
@(posedge clk_tb);
$display("Passed all tests");
$finish;
end

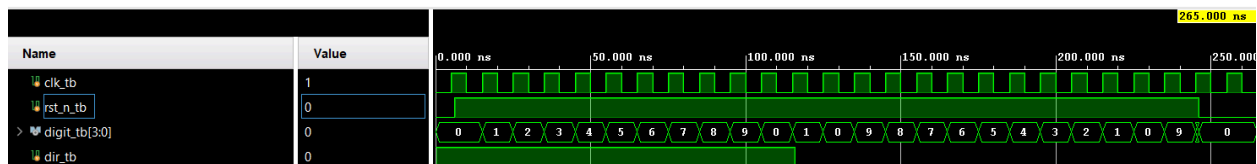
endmodule

```

Description:

To test the single digit up/down counter, I used an expected value and automatic checks to verify the counter increments and decrements properly with a rollover at 9 for incrementing and 0 for decrementing. I also tested that digit goes to zero on reset.

Waveform:



Output:

```

run 12 ms
Passed all tests
$finish called at time : 265 ns : File "D:/School/ECE3300/ECE3300_Lab6_GroupU

```

alu_tb.v:

```

module alu_tb(

);

    reg [3:0] a_tb;
    reg [3:0] b_tb;

    reg [1:0] ctrl_tb;

```

```

wire[7:0] result_tb;

integer i, j;

alu alu_tb(
    .a(a_tb),
    .b(b_tb),
    .ctrl(ctrl_tb),
    .result(result_tb)
);

initial
begin
    a_tb = 4'd0;
    b_tb = 4'd0;
    ctrl_tb = 2'b00;

    #1;
    if(result_tb !== 8'd0) begin // Test 1: Initialization
        $display("Fail: initialization test  a = %d, b = %b, result = %d", a_tb, b_tb, result_tb);
        $fatal;
    end
    for(i = 0; i < 16; i = i + 1) begin // Test 2: Addition
        a_tb = i;
        for(j = 0; j < 16; j = j + 1) begin
            b_tb = j;
            #1;
            if(result_tb !== ({4'b0000, a_tb} + {4'b0000, b_tb})) begin
                $display("Fail: addition test  a = %d, b = %d, result = %d", a_tb, b_tb, result_tb);
                $fatal;
            end
        end
    end
    end
    #1 ctrl_tb = 2'b01; // Test 3: Subtraction
    for(i = 0; i < 16; i = i + 1) begin
        a_tb = i;
        for(j = 0; j < 16; j = j + 1) begin
            b_tb = j;
            #1;
            if(result_tb !== ({4'b0000, a_tb} - {4'b0000, b_tb})) begin
                $display("Fail: subtraction test  a = %d, b = %d, result = %d", a_tb, b_tb, result_tb);
                $fatal;
            end
        end
    end
    end
    #1 ctrl_tb = 2'b10; // Test 4: Default control test
    #1 if(result_tb !== 8'd0) begin
        $display("Fail: default test  a = %d, b = %d, ctrl = %b, result = %d", a_tb, b_tb, ctrl_tb, result_tb);
        $fatal;
    end
    end

    #1 ctrl_tb = 2'b00;
    a_tb = 4'd5;

    #1 ctrl_tb = 2'b11;
    #1 if(result_tb !== 8'd0) begin
        $display("Fail: default test  a = %d, b = %d, ctrl = %b, result = %d", a_tb, b_tb, ctrl_tb, result_tb);
        $fatal;
    end
    end

```



```

        $display("Passed all tests");
        $finish;
    end

endmodule

```

Description:

To test the ALU, I used a nested for loop to automatically check every result for the 4-bit operands in both addition and subtraction. Additionally, I tested each control value to verify invalid values would result in the default case and set the result to zero

Output:

```

run 12 ms
Passed all tests
$finish called at time : 519 ns : File "D:/School/ECE3300/ECE3300_Lab6_GroupU

```

control_decoder_tb.v:

```

module control_decoder_tb(

);

    reg [3:0] inp_tb;
    wire [3:0] ctrl_tb;

    integer i;

    control_decoder ctrl_nibble_tb(
        .inp(inp_tb),
        .ctrl(ctrl_tb)
    );

    initial
    begin
        inp_tb = 4'd0;
        #1
        for(i = 0; i < 16; i = i + 1) begin
            inp_tb = i;
            #1;
            if(ctrl_tb != i) begin
                $display("FAIL: inp != op   inp = %b, ctrl = %b", inp_tb, ctrl_tb);
                $fatal;
            end
        end
        $display("Passed all tests");
        $finish;
    end

endmodule

```

Description:

To test the control decoder I used a for loop to test every 4-bit input and automatically verify the output nibble matches.

Output:

```
# run 1000ns
Passed all tests
$finish called at time : 17 ns : File "D:/School/ECE3300/ECE3300_Lab6_GroupU
```

seg7_scan_tb.v:

```
module seg7_scan_tb(

);

reg clk_tb;
reg rst_n_tb;

reg [7:0] result_tb;
reg [3:0] ctrl_nibble_tb;

wire [7:0] AN_tb;
wire [6:0] SEG_tb;

integer i;
integer tasksfailed;
integer tempfail;

reg [6:0] displaycheck;

seg7_scan scan_tb(
    .clk(clk_tb),
    .rst_n(rst_n_tb),
    .result(result_tb),
    .ctrl_nibble(ctrl_nibble_tb),
    .AN(AN_tb),
    .SEG(SEG_tb)
);

initial
begin
    clk_tb = 1'b0;
    rst_n_tb = 1'b0;

    result_tb = 8'h00;
    ctrl_nibble_tb = 4'h0;

    tasksfailed = 0;
    tempfail = 0;
    displaycheck = 7'b1111111;
end

always
```

```

begin
    #5 clk_tb = ~clk_tb;
end

initial
begin
    @(posedge clk_tb);
    #1;
    rst_n_tb = 1'b1;

    for(i = 0; i < 16; i = i + 1)
    begin
        result_tb = {i[3:0], i[3:0]};
        ctrl_nibble_tb = i[3:0];

        tempfail = tasksfailed;

        case(i)
            0: displaycheck = 7'b0000001;
            1: displaycheck = 7'b1001111;
            2: displaycheck = 7'b0010010;
            3: displaycheck = 7'b0000110;
            4: displaycheck = 7'b1001100;
            5: displaycheck = 7'b0100100;
            6: displaycheck = 7'b0100000;
            7: displaycheck = 7'b0001111;
            8: displaycheck = 7'b0000000;
            9: displaycheck = 7'b0001100;
            10: displaycheck = 7'b0001000;
            11: displaycheck = 7'b1100000;
            12: displaycheck = 7'b0110001;
            13: displaycheck = 7'b1000010;
            14: displaycheck = 7'b0110000;
            15: displaycheck = 7'b0111000;
            default: displaycheck = 7'b1111111;
        endcase

        #40;

        if(displaycheck !== SEG_tb)
        begin
            tasksfailed = tasksfailed + 1;
        end

        if(tasksfailed > tempfail)
        begin
            $display("SEG test failed at i = %d, SEG = %b, expected = %b", i, SEG_tb, displaycheck);
        end
    end

    if(tasksfailed == 0)
    begin
        $display("SEG tests passed");
    end
    else
    begin
        $display("%d SEG tests failed", tasksfailed);
    end

    #12000000;
    $finish;

```

```
end
```

```
endmodule
```

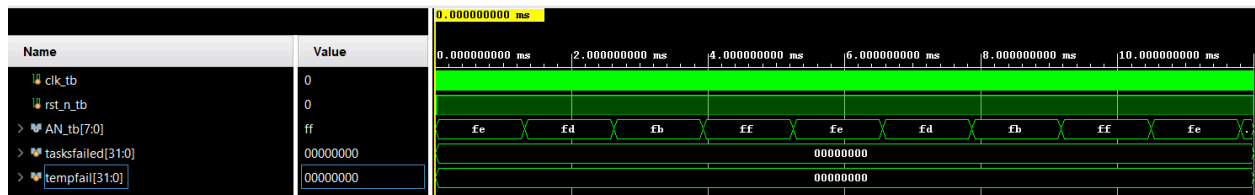
Description:

To test the 7segment display driver, each number is automatically checked against its corresponding digit pattern. After this, the testbench runs for 12ms to show every instance of AN display switching for scan multiplexing twice

Output:

```
run 12 ms
SEG tests passed
```

Waveform:



bcd_counter_alu_tb.v (top level)

```
module bcd_counter_alu_tb(
);
    reg [3:0] ctrl;
    wire [3:0] tens, ones;
    reg rst, clk;
    wire [7:0] result;

    bcd_counter ones_counter(.digit(ones), .dir(ctrl[3]),
        .rst_n(~rst), .clk(clk));
    bcd_counter tens_counter(.digit(tens), .dir(ctrl[2]),
        .rst_n(~rst), .clk(clk));
    alu add_sub(.a({4'b0, ones}), .b({4'b0, tens}),
        .ctrl(ctrl[1:0]), .result(result));

    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end

    integer i;
    initial begin
        ctrl = 4'b0;
        rst = 1'b0;
        for(i = 0; i < 10; i=i+1)begin
```

```

    #10;
end

ctrl[3] = 1'b1; //tens count down
rst = 1;
#10;
rst = 0;
for(i = 0; i < 10; i=i+1)begin
    #10;
end

ctrl[3:2] = 2'b01; //ones count down
rst = 1;
#10;
rst = 0;
for(i = 0; i < 10; i=i+1)begin
    #10;
end

ctrl[3:2] = 2'b11; //tens and ones count down
rst = 1;
#10;
rst = 0;
for(i = 0; i < 10; i=i+1)begin
    #10;
end

ctrl[1:0] = 2'b01; // subtract tens and ones
ctrl[3:2] = 2'b00; //tens and ones count up
rst = 1;
#10;
rst = 0;
for(i = 0; i < 10; i=i+1)begin
    #10;
end

ctrl[3] = 1'b1; //tens count down
rst = 1;
#10;
rst = 0;
for(i = 0; i < 10; i=i+1)begin
    #10;
end

ctrl[3:2] = 2'b01; //ones count down
rst = 1;
#10;
rst = 0;
for(i = 0; i < 10; i=i+1)begin
    #10;
end

ctrl[3:2] = 2'b11; //tens and ones count down
rst = 1;
#10;
rst = 0;
for(i = 0; i < 10; i=i+1)begin
    #10;
end

$finish;

```

end

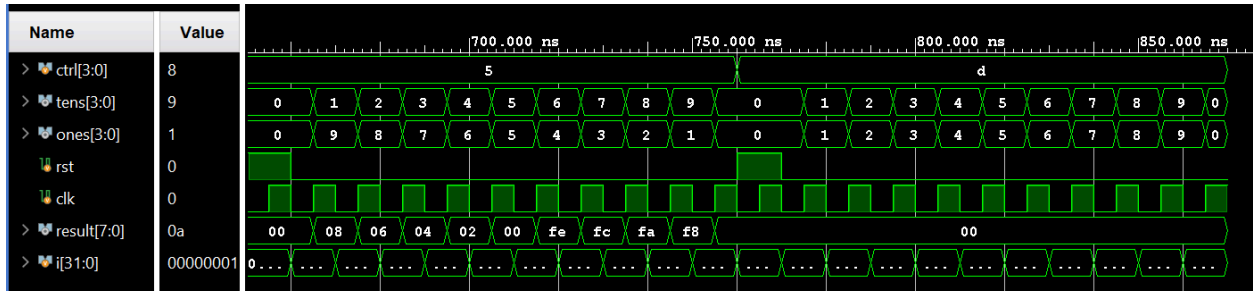
endmodule

Description:

This test bench tests the bcd counters and alu by running a full cycle of ten decimal digits for each possible control status. This tests the top level functionality while bypassing the display modules

Waveform:





Implementation:

Resource Utilization Report:

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	Bonded IOB (210)	BUFGCTRL (32)
top_lab6	36	62	1	34	1

Timing Report:

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 6.851 ns	Worst Hold Slack (WHS): 0.137 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 72	Total Number of Endpoints: 72	Total Number of Endpoints: 55
All user specified timing constraints are met.		

Group Video:

<https://youtu.be/xBECDBbRtbs>

Contributions:

Robert Stevenson: Testbenches, Core design
Ben Robles: Design/Implementation, Refined testbenches

Reflections:

Roberty Stevenson: In this lab I spent a lot of time writing a lot of code that did not end up in the final design or testbenches. This lab very much felt like an example of how sometimes productive effort does not always add linearly to the advancement of a project.

Ben Robles: This lab was very similar to the last but had a lot of parts to it. I liked working on the expanding implementation and having to adapt testbenches to adjustments in design